

```
Customer02 0.41 3.17 5.52
Customer03 0.44 5.48 7.86
Customer04 1.40 6.90 9.00
```

Here is another run and the counter is usually free:

```
1 counters with mean arrival and service times 1.5 2.0 minutes
Customer00 0.00 0.00 2.62
Customer01 1.67 0.96 3.99
Customer02 4.29 1.36 2.82
Customer03 7.38 0.00 1.02
Customer04 9.42 0.00 0.51
```

And here is an output with two counters:

```
2 counters with mean arrival and service times 1.5 2.0 minutes
Customer00 0.00 0.00 2.54
Customer01 0.67 0.00 2.01
Customer03 2.04 0.64 0.78
Customer04 2.04 0.77 1.93
Customer02 1.79 0.75 2.46
```

You will notice that the customers do not necessarily exit in the same order as they entered. Also, even with two counters, there are times when a queuing delay does occur.

Obviously, you need to experiment with a large number of customers. Besides, printing the results is not particularly useful or meaningful.

Statistical analysis of the simulation

You can collect the data in the process function and then use SciPy's statistical functions. SimPy includes some simple statistical methods, which cannot be explored here.

```
def process(name, time_in, wait_time, time_in_bank):
    wait_array.append(wait_time)
    tib_array.append(time_in_bank)
```

```
import scipy as np
wait_array=[]
tib_array=[]
main(1000, 100, 1.5, 2.0, 2)
wt=np.array(wait_array)
tib=np.array(tib_array)
print(15*' '+Mean Variance Median Max')
```

```
print('%s %6.2f %6.2f %6.2f %6.2f'%(Queue Time ',
    wt.mean(), wt.var(), np.median(wait_array), wt.max()))
print('%s %6.2f %6.2f %6.2f %6.2f'%(Time in Bank',
    tib.mean(), tib.var(), np.median(tib_array), tib.max()))
print('The percent of customers who did not have to wait %4.1f
    %%(100 - 100.0*len(wt.nonzero()[0])/len(wt)))
```

You keep appending times to the global Python lists during the simulation. The lists are converted to NumPy arrays and their mean, variance, median and maximum are printed. Of these, the median is not a property of a NumPy array and is computed using a standard NumPy method. With two counters, a customer is handled every minute; hence, it's also interesting to see how many customers did not have to wait. Numpy arrays have a number of useful properties—for example, to get indices of the elements that are not zero.

The results for a run with two counters will look like the following code:

```
2 counters with mean arrival and service times 1.5 2.0 minutes
Mean Variance Median Max
Queue Time 0.68 1.09 0.00 3.76
Time in Bank 2.55 2.99 2.11 9.48
The percent of customers who did not have to wait 58.0
```

With a sample of only a hundred, running the simulation again may give you substantially different results. You can experiment with larger sample sizes and explore the many more functions available in SciPy.

With this article, we come to the end of our exploration of SciPy, with the hope that you will find it useful when solving more complex research and educational problems. **END** 

References

- SimPy Tutorial: http://simpy.sourceforge.net/SimPyDocs/SimPy_Tutorials.html
- Erlang Distribution: http://en.wikipedia.org/wiki/Erlang_distribution
- SciPy Statistics: <http://docs.scipy.org/doc/scipy/reference/stats.html>

By: Anil Seth

The author works as a consultant. You can find him online at <http://sethanil.com> and reach him via email at seth.anil@gmail.com

Asia's Leading Open Source Magazine

To BUY or SUBSCRIBE, please visit www.linuxforu.com

www.linuxforu.com

